GOSIM
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM CHINA 2024

仓颉语言开源微服务框架 microservice开发实践

赛可出行|高级技术专家|赵刚



目 录
Contents

①项目起源：仓颉与微服务可能有哪些碰撞
②项目设计：仓颉版微服务的几大焦点设计
③使用实践：microservice探索与实践
④未来展望与规划



以前参与微服务工作

- C++微服务框架（bpe）

特点：XML流程同步编排 、 异步执行、C++插件

编号	增加feature	功能点
1	Session	分布式服务无状态
2	内置状态机	轻量易使用
3	语法扩展	提升表达能力
4	子过程	流程复用

- Java微服务框架（krpc）

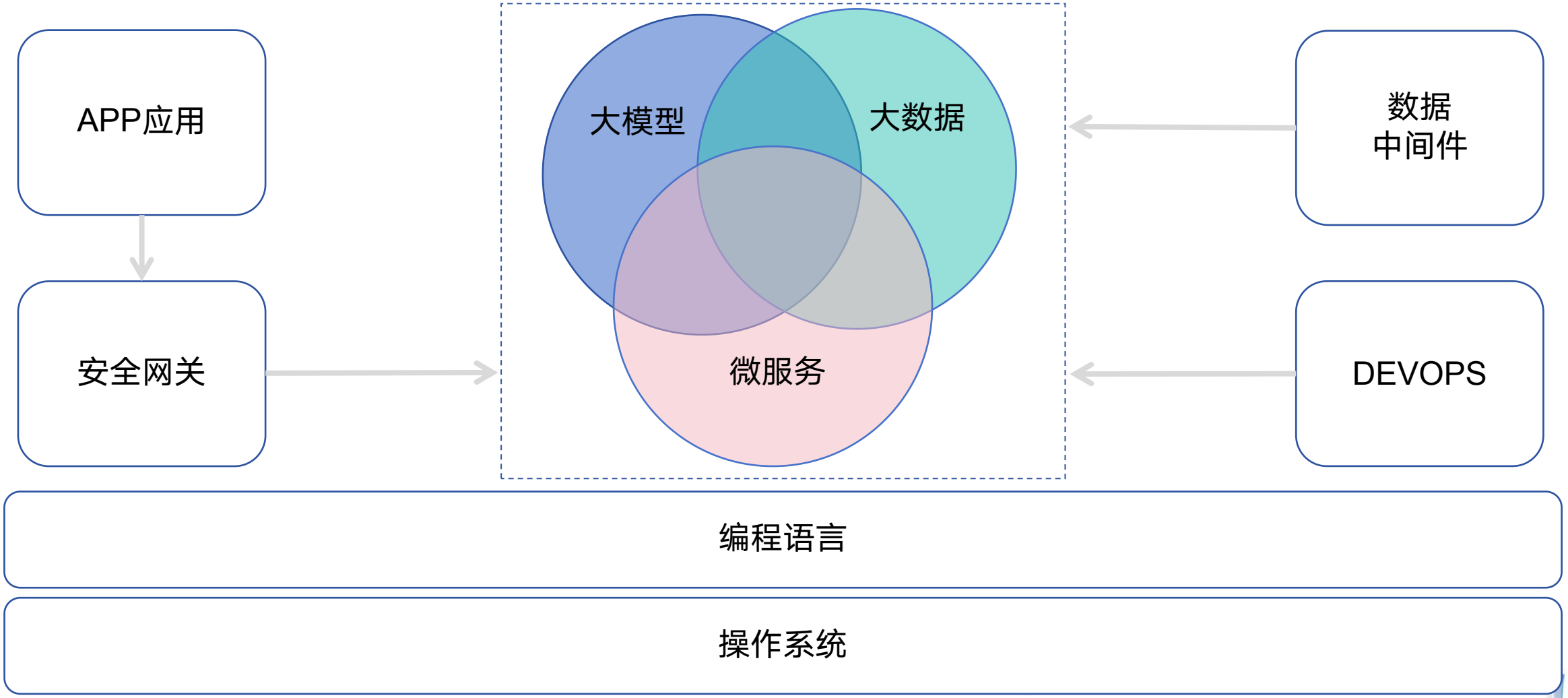
特点：同步和异步编写、高容错性

编号	增加feature	功能点
1	JDK8->JDK17	提升性能
2	接口契约验证	内外部契约变更感知
3	分布式性质判定	安全性、活性、公平性
4	知识图谱	微服务知识挖掘

研究领域：数据分析、服务治理、支付结算、金融科技



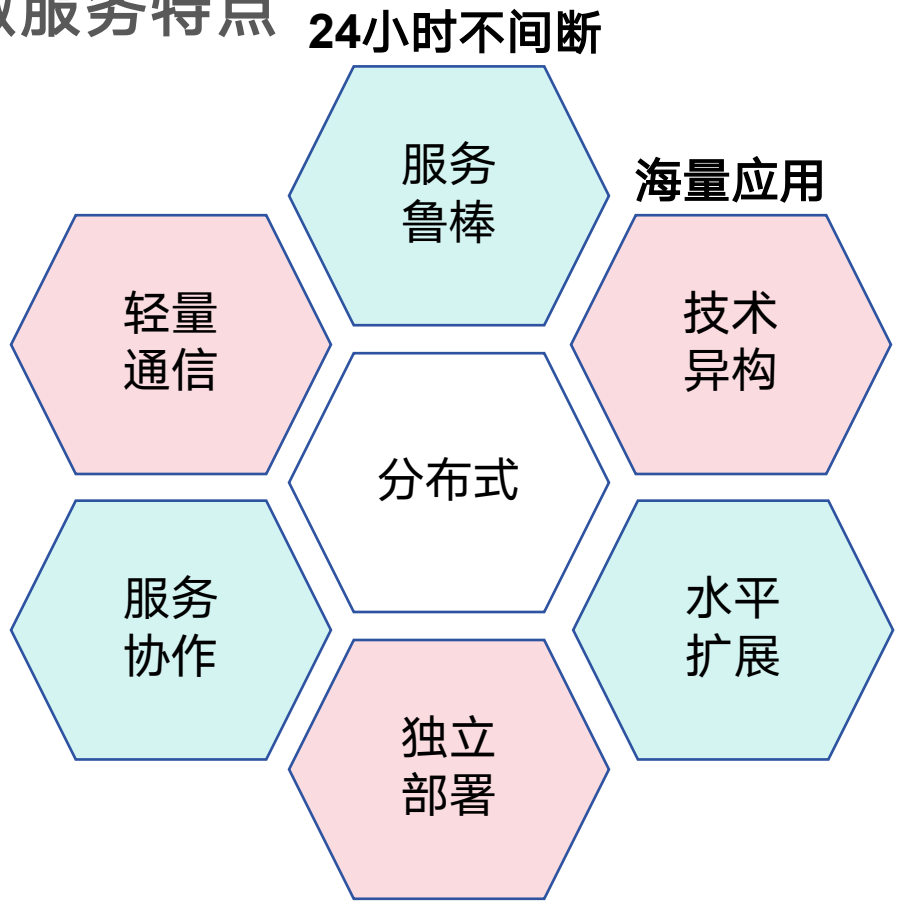
一 微服务体系



微服务特点与发展

GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

- 微服务特点



- 发展趋势（开放化+智能化）

① 技术融合
② 多语言支持
③ 开源生态
④ 安全与性能
⑤ 低代码平台
⑥ 产业数字化渗透
⑦ 服务标准化
⑧ AI集成

为什么选择仓颉语言开发微服务

GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

Q1.随仓颉语言发展，预计将涌现出一系列覆盖不同领域的专用库，如何集成到服务端？

- **优秀库集成**

开发一个全新服务框架，该框架将原生支持仓颉语言，从而避免了二次开发的繁琐过程

Q2.微服务的技术异构特性要求我们根据不同的技术栈来开发相应的微服务。那么，开发新服务，这种必要性有多大呢？

- **新语言特性**

按应用场景特点选择不同语言及技术栈，并采用轻量级协议和开放式的服务体系，可以构建出更加灵活、高效的微服务架构

Q3.随开发人员技能沉淀，有无必要提供对应的服务来支持？

- **开发人员友好**

参照nodejs，python等语言支持微服务，新语言的适用领域会扩大，对应技能的开发人员受益。



仓颉与微服务适配度

仓颉基础能力	适配度
类型安全	服务安全
异常机制	服务鲁棒
并发协程	开发高效
内存管理	维护简单
性能高效	节约资源
反射机制	可扩展性
语法丰富	简化开发

仓颉发展能力	适配度
表达能力	包含主流语言优点
云原生	启动快捷
跨语言调用	集成其他语言库

当前发挥 开发效率 + 应用性能

未来便利 灵活扩展 + 丰富生态



项目目标

- **轻量化** → 开发快捷、学习曲线低、无外部依赖
- **可扩展** → 对象和组件按需组装扩展、功能升级
- **易集成** → 容易与外部系统接口集成、组件集成
- **零侵入** → 不限制应用使用方式及响应报文格式

实现轻量仓颉语言版微服务，可以支持扩展升级，并能和其他微服务体系集成，不修改原应用协议



二 微服务设计-功能设计

GOSIM
GLOBAL OPEN-SOURCE INNOVATION MEETUP

1

WEB服务

- http
- rest
- json

01

2

注册发现

- 注册
- 发现
- 下线

02

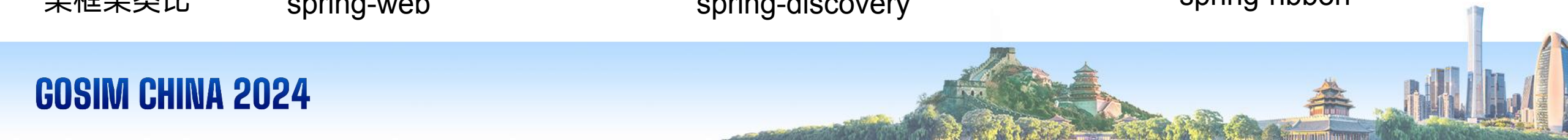
3

远程过程调用

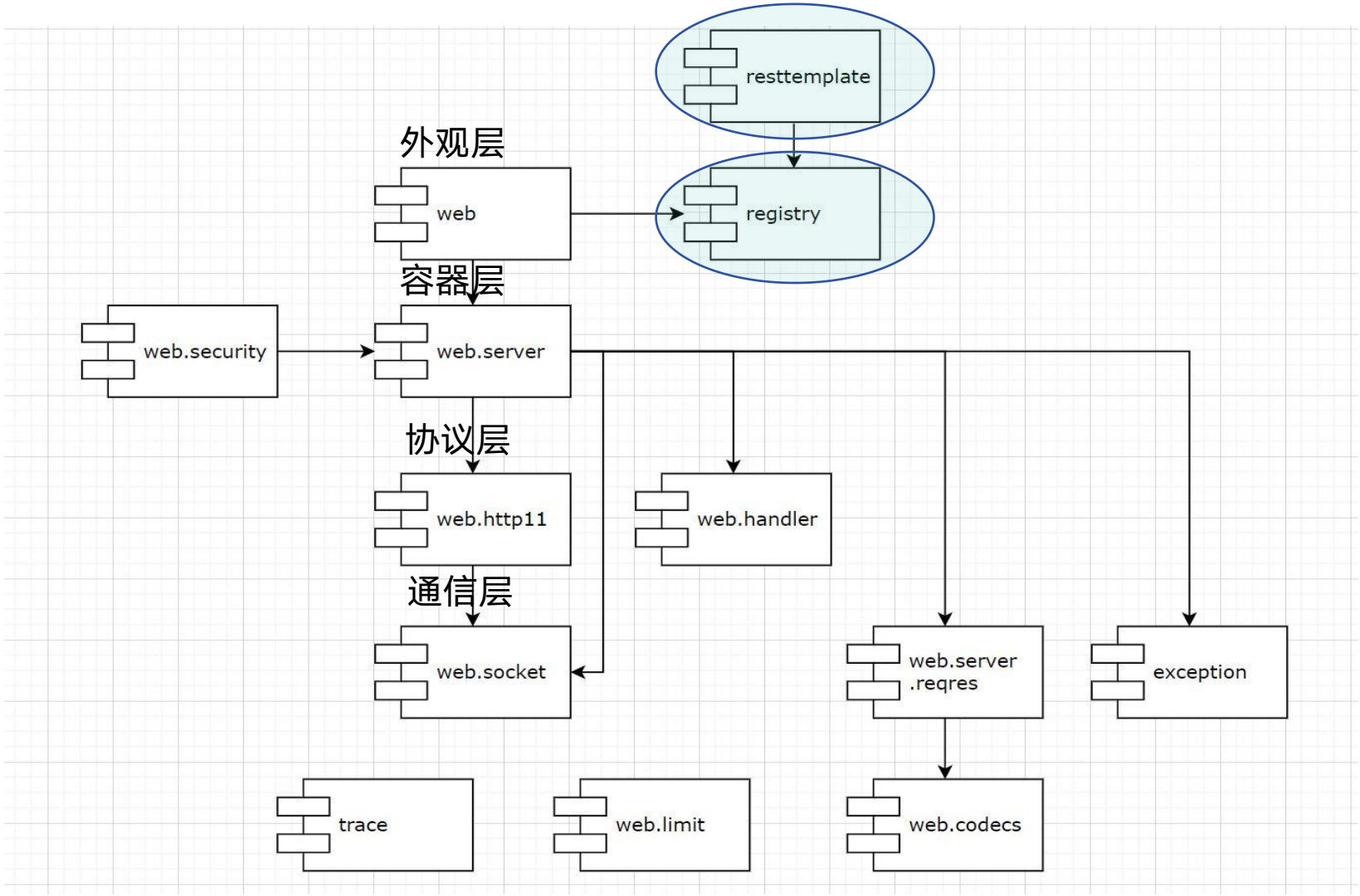
- 负载均衡
- 请求构造
- 响应解析

03

	通信协议	支持主流注册中心	服务互操作
某框架类比	spring-web	spring-discovery	spring-ribbon



微服务设计-包图设计

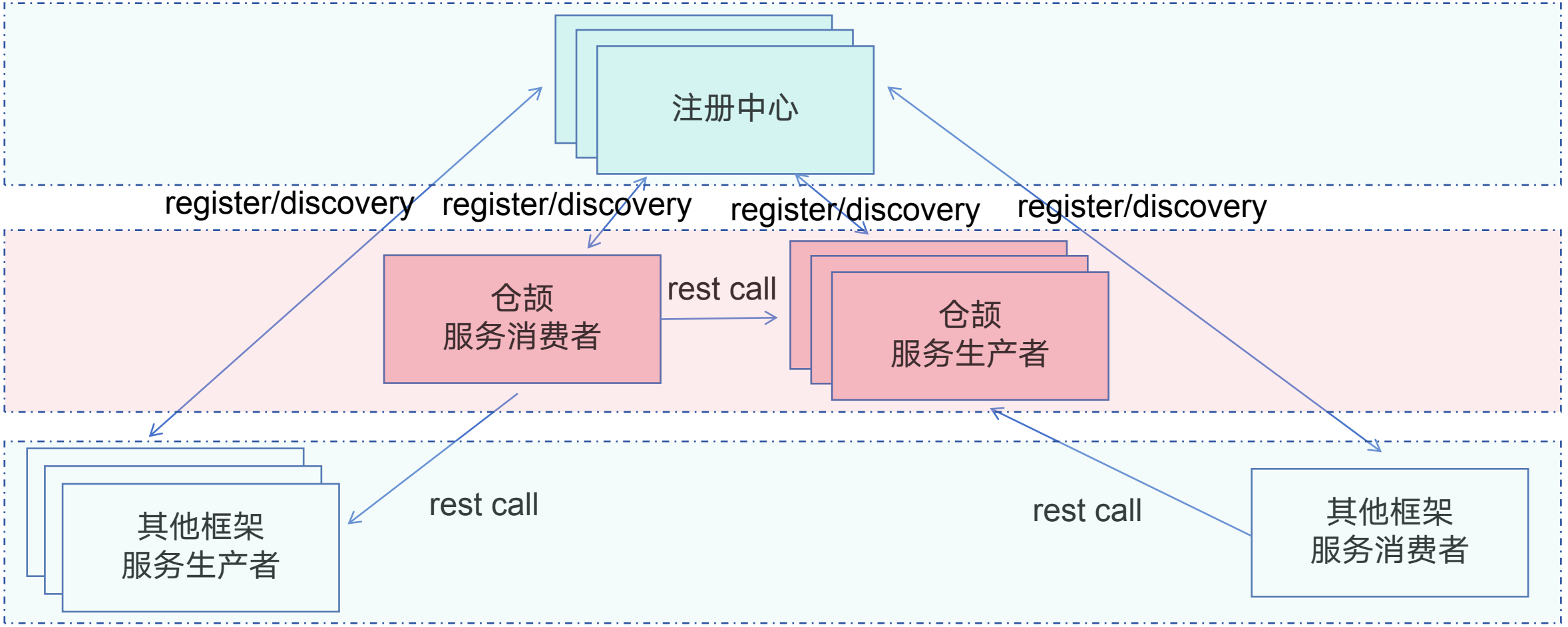

resttemplate
registry
外观层
web
容器层
web.security
web.server
协议层
web.http11
web.handler
通信层
web.socket
web.server
.reqres
exception
trace
web.limit
web.codecs

微服务设计-框架设计

编号	设计原则
1	开放适配原则
2	最小控制原则
3	最大兼容原则
4	现实思维原则
5	约定配置原则



微服务设计-1.开放适配原则



设计支持新仓颉服务和其他框架服务之间通过http协议互操作

微服务设计-2.最小控制原则

GOSIM
GLOBAL OPEN-SOURCE INNOVATION MEETUP

注册

```
public interface IRegistry {
  func config(cfg: Config): IRegistry
  func register(serviceName: String, port: Int64): Bool
  func deregister(serviceName: String, port: Int64): Bool
  func heartBeat(serviceName: String, port: Int64): Bool
  func discovery(serviceName: String): String
}
```

发现

```
interface IDiscovery {
  func find(serviceName: String): String
}
```

编解码

```
public interface ICodecs{
      func fromBody(body: String): Object
      func toBody(object: Object): String
      func getContentType(): String
}
```




微服务设计-3.最大兼容原则

GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

服务

```
public interface IServlet{
    func service(req: HttpServletRequest, res: HttpServletResponse): Unit
}
```

过滤器

```
public interface IFilter{
    func doFilter(req: HttpServletRequest, res: HttpServletResponse, filterChain: FilterChain): Unit;
}
```

异常处理器

```
public interface IExceptionHandler{
    func handle(req: HttpServletRequest, res: HttpServletResponse, e: Exception): Unit
}
```




微服务设计-4.现实思维原则

GOSIM
GLOBAL OPEN-SOURCE INNOVATION MEETUP

```
public func start():Unit{
    let socket = acceptor.accept();
    spawn{
        handler.run(socket);
    }

    spawn{
        start()
    }
}
```

1

```
public class SocketHandler {
    let dataprocessor: IDataProcessor
    public init(dataprocessor: IDataProcessor){
        this.dataprocessor = dataprocessor
    }

    public func run(socket: Socket){
        let wrapper = SocketWrapper(socket)
        var ret = true
        while(ret){
            ret = dataprocessor.process(wrapper);
        }
        wrapper.close();
    }
}
```

2

```
public class Http11DataProcessor <: IDataProcessor{
    var data = ArrayList<Char>()
    var dispatcher: IDispatcher;
    public init(dispatcher: IDispatcher){
        this.dispatcher = dispatcher
    }

    public func process(wrapper: SocketWrapper): Bool {
        var result = innerProcess(wrapper)
        return result
    }
```

3



微服务设计-5.约定配置原则

resources > application.properties

```
server.name=hello
server.port=8080
server.registryAddress=http://localhost:8761/eureka/apps
server.maxConnections=2000
server.acceptCount=200
server.uriEncoding=UTF-8
server.maxHttpHeaderSize=8192
server.maxHttpBodySize=81920
server.connectionTimeout=3000
server.logging.level=info
server.refer=provider01,provider02
```

提升可读性、提高配置灵活性



三 开发实践

编号	WEB容器	服务集成方式	特点
1	内置WEB容器	自动装配	使用完整框架
2	内置WEB容器	手动装配	支持自定义组件
3	外部WEB容器	手动装配	支持使用其他框架的WEB容器



1.MicroBoot（自动装配）

```
class BootServlet <: IServlet{
    public func service(req:HttpServletRequest,res:HttpServletResponse){
        println("BootServlet called")
        var root = ResponseJsonObject()
        root.put("code", 0)
        root.put("msg", "success")
        var data = ResponseJsonObject()
        data.put("uid","123456")
        root.put("data",data)
        res.setBody(root)
    }
}
```

```
main(): Unit{
    println("hello microbootServer")
    microbootServer()
    sleep(86400*365*Duration.second)
}
```

```
func microbootServer(){
    let micro = MicroBootApplication()
    micro.getRouter().get("/fo", BootServlet())
    micro.run()
    //下面代码发起一次调用测试
    println("do resttemplate call response="+micro.getRestTemplate().getForEntity("http://hello/fo",""))
}
```

自动装配（从配置文件读取）

- MicroBootApplication //定义启动类
- Url Router //定义路由器规则



2.ManualServer（手工装配）

```
class DemoServlet <: IServlet{
    public func service(req:HttpServletRequest,res:HttpServletResponse){
        res.setBody("it is only a demo")
    }
}
```

```
main(): Unit{
    println("hello manualServer")
    manualServer()
    sleep(86400*365*Duration.second)
}
```

```
func manualServer(){
    var cfg = Config()
    cfg.url = "http://localhost:8761/eureka/apps"
    let rgs = EurekaRegistry().config(cfg)
    let ws = Application().name("hello").port(8000).registry(rgs);
    ws.getRouter().get("/fo", DemoServlet())
    ws.run()
    //下面代码发起一次调用测试
    spawn { =>
      sleep(10*Duration.second)
      let resttemplate = RestTemplate(rgs)
      println(resttemplate.getForEntity("http://hello/fo",""))
    }
}
```

手工装配组件（不用配置文件）

- Application //定义应用服务
- Registry //定义注册中心
- RestTemplate //定义客户端远程过程调用器
- Url Router //定义路由器规则

3.ExternServer（外部web容器）

```
var foHandler = FuncHandler(
    {
        context => context.responseBuilder.body("hello world")
    }
)
```

```
main(): Unit{
    println("hello externServer")
    externServer()
    sleep(86400*365*Duration.second)
}
```

```
func externServer(){
    var cfg = Config()
    cfg.url = "http://localhost:8761/eureka/apps"
    let rgs = EurekaRegistry().config(cfg)
    let resttemplate = RestTemplate(rgs)
    let ws = WebServer(8000, 3*Duration.second, 5*Duration.second)
    .setAppName("hello")
    .handle("/fo",foHandler)
    .registry(rgs);
    ws.start()
    //下面代码发起一次调用测试
    spawn { =>
      Common.sleeps(6*Duration.second)
      println(resttemplate.getForEntity("http://hello/fo",""))
    }
}
```

使用外部web容器

仅使用提供的注册发现以及远程过程调用功能，这里是演示使用仓颉内置的web server

4.Misc(常见功能)

GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

- **自定义异常处理**

```
class NoMappingHandler <: IExceptionHandler{
  public func handle(req: HttpServletRequest, res: HttpServletResponse, e: Exception): Unit{
    res.setBody("No URL Match")
    res.setStatus(500)
  }
}
```

- **自定义过滤器**

```
public class MyFilter <: IFilter{
    public func doFilter(req: HttpServletRequest, res: HttpServletResponse, filterChain: FilterChain): Unit{
        filterChain.doFilter(req, res)
    }
}

FilterChain.addFilterToHead(MyFilter())
```

- **跨域设置**

```
var crossOrigin = CrossOriginFilter()
crossOrigin.add("/api/test")

FilterChain.addFilterToHead(crossOrigin)
```




四 项目规划

基础功能

http1.1	dynamic route	encoder/decoder	filter
exception handler	cookie	cross domain	log

扩展功能

session	distributed trace	https/http2	csrf
multiple port	failover	circuit breaker	rate limiter
validator	retry	metrics	failback
mock	health check	api gateway	thread pool sepearter



GOSIM
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM CHINA 2024

开源仓库地址：https://gitcode.com/Cangjie-TPC/**microservice**

欢迎开源爱好者多提意见以及项目共建

THANK YOU

扫一扫上面的二维码图案，加我为朋友。

个人微信号交流

